

AgenticSys v2 — System Design Summary

2026-05-06

AgenticSys v2 — System Design Summary

Combines: tool-calls-and-redaction, team-construction, data-query (2026-05-06)

Part I: Tool Calls, Redaction, Firewall & Skill Loading

1. The Core Problem

LLMs can only generate text. They cannot browse the web, query a database, or check live data. **Tool calls** are the bridge between “text generation” and “real-world actions.” The LLM never runs code itself — it *requests* that your code runs something on its behalf.

2. The Mental Model: A Request–Execute–Report Loop

Client code

```
|
+- defines tool schemas -----+
+- sends [messages + tools] to API --> LLM reasons; picks tool |
|                                     |
| <-- tool_call response -----+
|
+- executes real function(s)
+- appends tool result to messages
+- sends back to API --> LLM sees result -> writes final answer |
|
~
+-- loop repeats if LLM needs more tools -----+
```

Think of it as an assistant who can ask you to look things up:

```
You -> LLM: "What's the weather in NYC?"
LLM -> You: "Please call get_weather('NYC') and give me the result."
You -> (executes get_weather) -> "72 degrees F, sunny"
You -> LLM: "Tool result: 72 degrees F, sunny"
LLM -> You: "It's 72 degrees F and sunny in New York City!"
```

3. The Mechanism — Five Steps

Step 1 — Define tools (JSON schema, not code)

```
tools = [{
  "type": "function",
  "function": {
    "name": "get_weather",
    "description": "Get current weather for a city",
    "parameters": {
      "type": "object",
      "properties": {
        "city": {"type": "string"}
      },
      "required": ["city"]
    }
  }
}]
```

This is a *description* — the LLM reads it to know what capabilities exist. No code runs here.

Step 2 — LLM responds with a `tool_calls` block (not text)

```
{
  "role": "assistant",
  "tool_calls": [{
    "id": "call_abc123",
    "type": "function",
    "function": {
      "name": "get_weather",
      "arguments": "{\"city\": \"NYC\"}"
    }
  }]
}
```

Instead of answering directly, the model emits a structured request. Arguments are a JSON string — typed, validated, safe to parse.

Step 3 — Your code executes the real function

```
if response.choices[0].message.tool_calls:
    for tc in response.choices[0].message.tool_calls:
        result = get_weather(json.loads(tc.function.arguments)["city"])
```

Step 4 — Return the result as a tool role message

```
messages.append({
    "role": "tool",
    "tool_call_id": "call_abc123",
    "content": "72 degrees F, sunny"
})
```

Step 5 — Call the API again; LLM produces the final answer

The LLM now has the data it needed and writes a normal text response.

4. Key Design Choices

Design choice	Reason
LLM outputs a schema request, not executable code	Sandboxing — the model never runs arbitrary code
Tool result sent as a new conversation turn	Keeps full history; model sees all context
<code>tool_call_id</code> links request to result	One response can request multiple tools in parallel
Arguments are a JSON string	Typed, validated input; prevents injection

5. The Redaction Step

In production (SafeChain path), **every message is sanitized before it leaves the client** — before hitting the firewall, before the LLM sees it.

What it does

Located in `llm/firewall_stack.py` -> `sanitize_message()` and applied by `llm/safechain_client.py` -> `_redact_message()`:

```
_CASE_ID_RE = re.compile(r"CASE-\d+")
_DIGIT_RUN_RE = re.compile(r"\d{6,}")

def sanitize_message(message: str) -> str:
    masked = _CASE_ID_RE.sub("[CASE-ID]", message)
    return _DIGIT_RUN_RE.sub("***MASKED***", masked)
```

Pattern	Masked as	Example
CASE-123456	[CASE-ID]	Case identifiers
Any 6+ digit run	***MASKED***	Account numbers, SSNs, phone numbers

Where it sits in the pipeline

```

Agent / SDK
|
|  messages = [system, user, tool_result, ...]
|
v
_redact_message()      <- sanitize every outbound message
|
|  long digits -> ***MASKED***
|  CASE-\d+    -> [CASE-ID]
|
v
_combine_messages()    <- flatten multi-turn -> single string
|                      (SafeChain: only one human message)
|
v
SafeChain Firewall     <- HTTP POST; firewall evaluates content
|
v
LLM                    <- sees only masked identifiers

```

Why before flattening?

1. **No PII leaks through concatenation** — joining messages before masking could create new 6-digit sequences at join boundaries.
2. **Retry safety** — `_inject_guidance()` also re-redacts every message when it appends firewall guidance on a retry, so redaction is idempotent across attempts.

The `redact_payload` helper (deep recursion)

For tool *results* (Pydantic models, dicts, lists), `firewall_stack.redact_payload()` walks the entire structure:

```
def redact_payload(payload):
    if isinstance(payload, str):    return sanitize_message(payload)
    if isinstance(payload, dict):   return {k: redact_payload(v) ...}
    if isinstance(payload, list):   return [redact_payload(v) ...]
    if isinstance(payload, BaseModel):
        return Type.model_validate(redact_payload(payload.model_dump()))
    return payload
```

Even nested tool outputs — e.g., a `SpendingReport` with embedded account numbers — are fully masked before the LLM reads them.

6. Firewall Handling

After redaction, every API call passes through a firewall layer.

Implementation	File	Used when
FirewalledAsyncOpenAI	llm/firewall_client.py	Dev / direct OpenAI path
SafeChainChatCompletions	llm/safechain_client.py	Prod / SafeChain path

`FirewallStack` owns the shared state: `logger`, `max_retries=2`, and `asyncio.Semaphore(concurrency_cap=3)`.

The retry-with-guidance loop

```
messages = [_redact_message(m) for m in messages]    # 1. pre-redact
attempt = 0
while True:
    try:
        async with firewall.semaphore:                # 2. acquire slot
            return await base.create(messages=messages, ...)
    except FirewallRejection as e:
        if attempt >= firewall.max_retries:
            raise                                       # 3a. give up
        attempt += 1
        messages = _inject_guidance(messages)         # 3b. inject hint, retry
```

`_inject_guidance` re-redacts all messages and appends `FIREWALL_GUIDANCE` to the system message so the LLM knows why it was blocked.

HTTP error mapping (SafeChain path)

HTTP status	Meaning	Action
401	Token expiry	Refresh model; retry once immediately (outside the loop)
403	Firewall block	<code>raise FirewallRejection("403", ...)</code>
400	Bad request	<code>raise FirewallRejection("400", ...)</code>

Firewall flow

```

messages (redacted)
|
| async with semaphore      <- cap concurrent calls
v
API call
|
+-- success --> return ChatCompletion
|
+-- FirewallRejection
|
|   +-- attempts < max_retries?
|   |   |
|   |   yes --> _inject_guidance() --> retry
|   |
|   +-- no --> log firewall_blocked --> raise

```

7. Combined Picture (SafeChain prod path)

1. User question arrives
2. Orchestrator composes messages + tool schemas
3. -- REDACTION -- every message sanitized (digits, case IDs masked)
4. Messages flattened into one string (SafeChain: one human message)
5. SafeChain firewall evaluates; blocks or passes
6. LLM emits {"tool_call": {...}} JSON (no native function-calling)
7. SafeChain shim parses JSON -> synthesizes OpenAI tool_calls object
8. Agent SDK dispatches real tool (data query, etc.)
9. Tool result -> -- REDACTION -- redact_payload() on the result struct
10. Result appended as "Tool result:" label in next flattened message
11. Steps 4-10 repeat until LLM emits {"output": {...}}
12. Final answer returned to user

8. How Markdown Skills Are Loaded

Every skill is a `.md` file with a YAML frontmatter block and a body. The frontmatter declares metadata; the body becomes part of an agent's system prompt or a tool description.

Frontmatter fields:

Field	Values	Meaning
<code>name</code>	string	Human-readable identifier
<code>description</code>	string	Used as the tool description when <code>mode=tool</code>
<code>type</code>	<code>workflow</code> , <code>domain</code> , <code>helper</code>	Folder the file lives under
<code>owner</code>	list of agent names	Which agents claim this skill
<code>mode</code>	<code>inline</code> , <code>tool</code>	How the body reaches the LLM

`load_skill(path)` parses the frontmatter fence, `yaml.safe_loads` it, separates reserved keys from `meta`, and returns a typed `Skill` object. `load_skills_for(agent_name)` rglobs the tree and filters by `owner`.

Two delivery modes

mode: inline — body concatenated into the system prompt via `render_inline_prompt()`. The LLM reads it as standing instructions on every turn.

mode: tool — body set as `wrapper.__doc__` in `agent_factories/helper_tools.py` via `_with_doc(fn, skill_body)`. `bind_tools()` reads `__doc__` and uses it as the tool description in the API request.

End-to-end skill loading picture

```

skills/workflow/team_construction.md
|
| load_skill()          <- parse frontmatter + body
v
Skill.body (plain string)
|
| _compose_orchestrator_instructions()
v
"=== Team Construction ===\n<body>\n\n---\n\n=== Data Catalog ===\n..."
|
| Agent(instructions=...)
v
System prompt <-- LLM reads this on every turn

```

For tool-mode skills:

```

Skill.body
|
| _with_doc(acropedia_lookup, body)
v
wrapper.__doc__ = body
|
| model.bind_tools([wrapper, ...])
v
Tool description in the API request <-- LLM reads this when deciding
                                         whether to call the tool

```

Redaction layer breakdown

Layer	What it does	When it runs
<code>redact.md</code> (inline skill)	Instructs the LLM to mask in its own output	At generation time
<code>sanitize_message</code> (Python)	Masks inbound text before it reaches the LLM	Before every API call
<code>redact_payload</code> (Python)	Masks structured tool results recursively	After every tool execution

9. When Does the Specialist See the Tool Description?

The specialist is built with `data_query.md` as its system prompt and the `@function_tool` callables as its `tools=` list. Both reach the LLM in **every API request, simultaneously**.

Request structure

```

API request
+-- messages[0] (system)
|   +-- _compose_instructions()
|       +-- data_query.md body          <- tool guidance prose
|       +-- Domain: <skill.name>
|       +-- Data hints, risk signals, ...
|
+-- tools: [
    { name: "query_table",
      description: "<wrapper __doc__>", <- @function_tool schema
      parameters: { ... } },
    { name: "aggregate_column", ... },
    ...
  ]

```

Strategic vs. tactical split

Source	Layer	Purpose
<code>data_query.md</code> body in system prompt	Strategic	Routing rules: when to use each tool, ordering
<code>@function_tool</code> wrapper	Tactical	Argument shapes, parameter constraints
<code>__doc__</code> in tools array	Developer only	Internal logic, never sent to the API
<code>_xxx_impl</code> docstring		

Both the system prompt and the tools array are billed as input tokens on every turn — before a word of user question is counted.

10. Summary Table

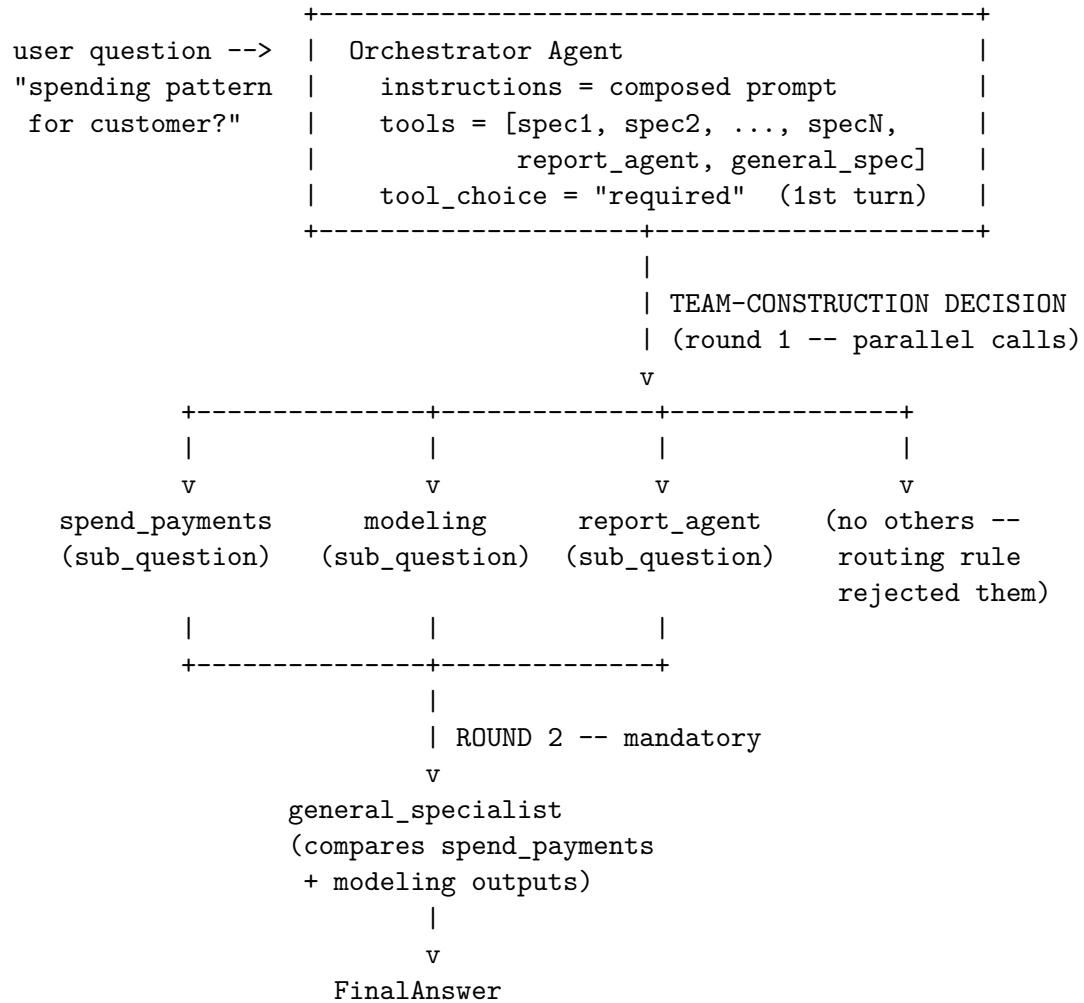
Concept	One line
Tool call	LLM <i>requests</i> your code to run something; you run it and report back
Tool schema	JSON description of a function's name, params, and purpose
<code>tool_call_id</code>	Ties each result to the specific request that triggered it
Redaction step	PII masking applied to every message before the firewall sees it
<code>sanitize_message</code>	Replaces 6+ digit runs and <code>CASE-\d+</code> tokens with masked placeholders
<code>redact_payload</code>	Recursively sanitizes structured tool outputs (dicts, Pydantic, lists)
SafeChain constraint	No native tool-calling; schemas injected as text; JSON response parsed back
<code>FirewallStack</code>	Shared state: logger, <code>max_retries</code> (2), concurrency semaphore (cap 3)
<code>FirewallRejection</code>	Typed exception carrying HTTP code + message; triggers retry-with-guidance
<code>_inject_guidance</code>	Re-redacts all messages; appends <code>FIREWALL_GUIDANCE</code> to system message
Concurrency cap	Semaphore limits parallel in-flight requests to avoid rate exhaustion
Skill file	<code>.md</code> with YAML frontmatter (name, type, owner, mode) + instruction body
<code>load_skill</code>	Parses one <code>.md</code> into a typed <code>Skill</code> object (frontmatter + body)
<code>mode: inline</code>	Skill body concatenated into the agent's system prompt
<code>mode: tool</code>	Skill body attached as <code>__doc__</code> ; LLM sees it as a tool description
<code>data_query.md</code> in system prompt	Strategic layer: routing rules for when and why to call each tool
<code>@function_tool</code> wrapper <code>__doc__</code>	Tactical layer: argument shapes and parameter constraints
<code>_impl</code> docstrings	Developer-only documentation; never sent to the API

Part II: Team Construction — Spending Pattern Walkthrough

This section walks the **team-construction** decision: how the orchestrator picks which specialists to call, frames each one's sub-question, and enforces the multi-round protocol that ends in a **FinalAnswer**. It uses the spending-pattern question as the running example.

Team construction is **not** a separate stage with its own JSON output. It IS the orchestrator's first turn — the decision is *expressed* by which tool calls the orchestrator emits, in what shape, and in parallel.

11. Where Team Construction Happens



Two passes through the orchestrator LLM:

1. **Round 1:** emit a parallel tool-call batch — *N domain specialists
 - `report_agent*`. The team is built here.
2. **Round 2:** emit `general_specialist(...)` (mandatory whenever round 1 used 2+ domain specialists).

Only after round 2 is the orchestrator allowed to emit a `FinalAnswer`.

12. What the Orchestrator Sees at Decision Time

`_compose_orchestrator_instructions` joins five blocks:

1. `skills/workflow/team_construction.md` — routing rules
2. `skills/workflow/data_catalog.md` — table-level grounding
3. `skills/workflow/synthesis.md` — downstream merge contract
4. `skills/workflow/balancing.md` — report vs. team reconciliation
5. **TOOL-USE DISCIPLINE block** — the hard protocol gates

Then optionally: pillar `concept_glossary` and the auto-generated **TEAM ROSTER** (`_render_team_roster`), which for each specialist surfaces its `description`, `data_hints` (table ownership), and top 3 `risk_signals`. Without this roster the orchestrator would route blind.

A second enrichment hits each specialist's tool-description string via `_describe_specialist(...)`:

Domain specialist 'spend_payments'. Spend & Payments -- payment trends, delinquency, spend spikes. Owns data tables: `txn_monthly`, `spends`, `payments`. Call with a focused sub-question scoped to this domain.

The TOOL-USE DISCIPLINE block — hard gates

Three rules from `orchestrator_agent.py:73-108`:

1. **Grounding gate** — every `FinalAnswer` MUST be preceded by (a) at least one `report_agent` call AND (b) at least one domain specialist.
2. **Parallel execution** — `report_agent` and every selected domain specialist must be emitted in a SINGLE response.
3. **General-specialist gate** — when 2+ domain specialists are on the team, `general_specialist` MUST be called in round 2 after their results land.

What's in the tools array

```
tools = [redacting_tool(s, name=s.name, description=_describe_specialist(s))
          for s in specialists]
tools.append(redacting_tool(report_agent, ...))
tools.append(redacting_tool(general_specialist, ...))
```

`tool_choice="required"` forces the first orchestrator turn to be a tool call. The SDK default `reset_tool_choice=True` flips this back to "auto" after the first call so synthesis is unblocked.

13. The Routing Decision for “Spending Pattern”

The cross-domain row in `team_construction.md:31` is the deciding rule:

spending / spend pattern / spend behavior / spend trajectory / spend volume / merchant concentration

MUST include BOTH `spend_payments` AND `modeling` (+ `crossbu` only when the question is explicitly B2B).

Three consequences:

- **Team size = 2 specialists.** The cross-domain table overrides the “1-2 is normal” default — `spend_payments` alone is incomplete.
- **crossbu is excluded.** The question doesn’t mention B2B/charge volume; the edge-case block reinforces this (`balance != spend`).
- **Other specialists dropped.** Selection rule #2 forbids “for context” picks.

Sub-question framing per specialist

Specialist	Framing principle	Example sub-question
spend_payments	Use <code>spends_data</code> and <code>payments</code> vocabulary; ask the broad pattern question because the domain skill’s playbook already covers temporal shape + merchant concentration + spend-vs-payment ratio.	<i>“Walk through the customer’s spending pattern: monthly volume + count, top recurring and high-value merchants by name, industry mix, per-merchant trends, and spend-vs-successful-payments ratio.”</i>
modeling	Use <code>model_scores</code> and <code>score_drivers</code> vocabulary; ask what the ML scores see.	<i>“What do the internal model scores say about this customer’s spending pattern? Cover spend-related drivers, score evolution over the spend window, divergence between model and bureau.”</i>
report_agent	Curated-report lookup; near-verbatim of the root.	<i>“Pull any prior curated narrative about the customer’s spending pattern, recurring merchants, or unusual spend behavior.”</i>

The framings are non-overlapping: `spend_payments` works the transaction tape; `modeling` works the ML signal; `report_agent` works the prior narrative. Selection rule #4 (“orthogonal across specialists”) is the formal source of this discipline.

14. Round 2 — `general_specialist`

After all three round-1 results land, the hard gate fires. Because the team had 2+ domain specialists, the orchestrator **MUST** emit:

```
general_specialist(sub_question="<spend_payments findings>;
                               <modeling findings>;
                               compare and surface contradictions")
```

`general_specialist` is tool-less (`tools=[]`) — pure synthesis on in-context outputs. It focuses on: does `modeling`’s out-of-pattern spend signal align with the spend spike `spend_payments` flagged? If contradictions exist, they must appear in the orchestrator’s **flags**.

15. Compliance Self-Check Before `FinalAnswer`

```
count unique domain specialists called this turn -> N
if N >= 2:
    scan conversation for a general_specialist tool result
    if absent: call general_specialist FIRST, then FinalAnswer
    if present: proceed to FinalAnswer
if N == 1:
    skip general_specialist
    proceed to FinalAnswer
```

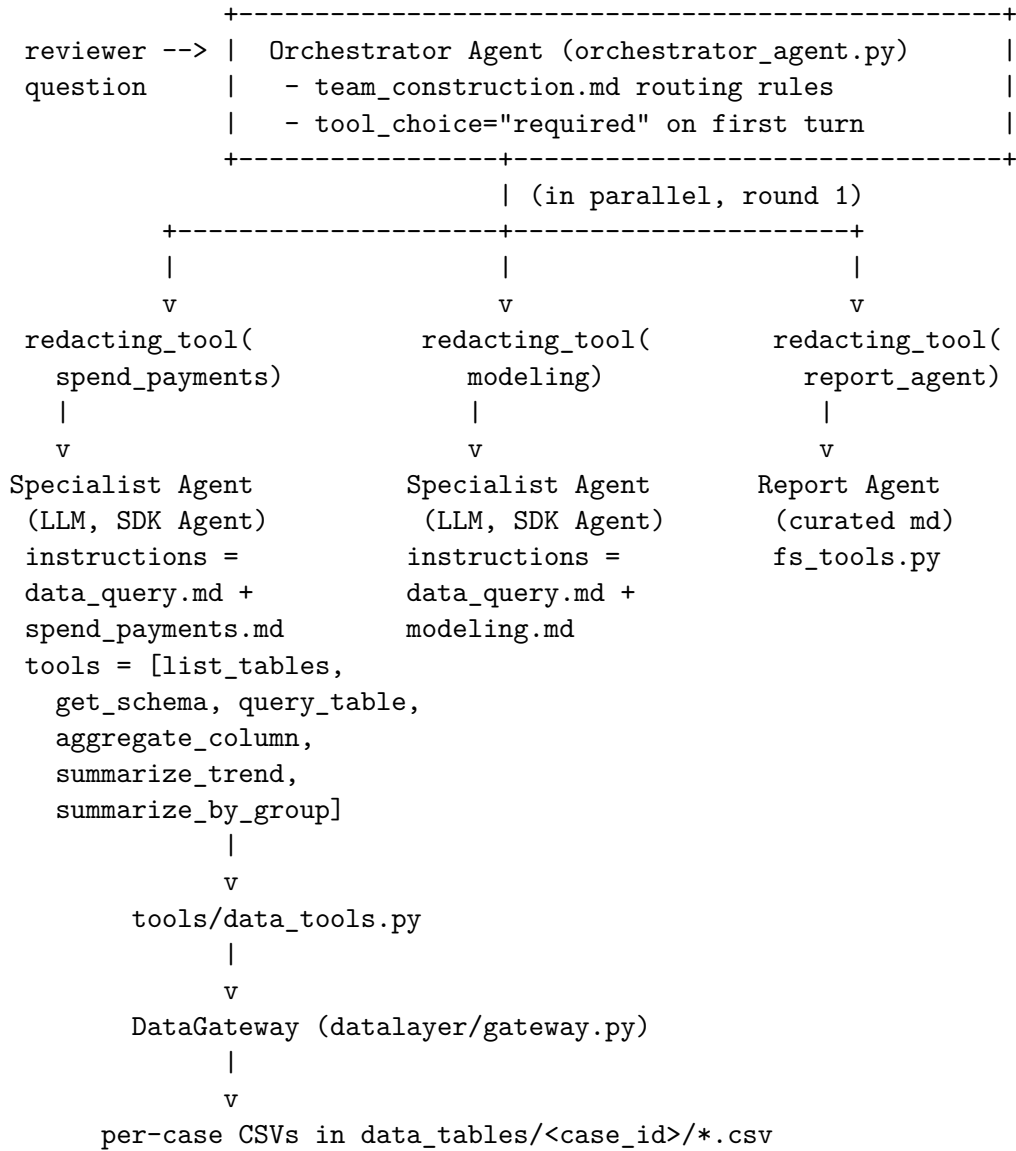
16. What Can Go Wrong

Failure	Caught by
Routing only <code>crossbu</code> for a spending-pattern question Answering with <code>spend_payments</code> alone (omitting <code>modeling</code>)	<code>team_construction.md:39</code> edge-case block + cross-domain table forcing <code>spend_payments</code> Cross-domain row explicitly calls this incomplete; selection rule #1 says read this table BEFORE applying defaults
Emitting <code>FinalAnswer</code> after only one specialist (no <code>report_agent</code>) Skipping <code>general_specialist</code> after 2 domain specialists Adding <code>bureau</code> “for context”	TOOL-USE DISCIPLINE rule #1 — both required Compliance self-check at <code>orchestrator_agent.py:99-107</code> Selection rule #2 — every pick must carry weight
Reshuffling the team for a follow-up	Selection rule #4 — reuse prior team so per-specialist memory is preserved
Hallucinating a <code>FinalAnswer</code> from prompt text alone	<code>tool_choice="required"</code> forces a tool call on the first turn

Part III: Data Query — Spending Pattern Walkthrough

This section walks the full path a reviewer's question takes through the v2 agent graph. Every callout is anchored to a file and line.

17. The Wire at a Glance



18. Build-Time Wiring

The graph is constructed once per session by `Orchestrator.__init__`:

1. **Domain skills discovered** — `list_domain_skills()` globs `skills/domain/*.md`. Relevant ones: `spend_payments.md`, `modeling.md`.
2. **Specialists built** — `build_specialist_agent(skill, pillar, model)` creates an `agents.Agent` whose instructions concatenate: `data_query.md` body + domain skill body + hints/signals + pillar config. `tool_choice="required"` forces the first turn to be a tool call.
3. **Report and general specialists built** — curated-md reader and cross-domain comparator.
4. **Orchestrator assembled** — each specialist is wrapped as a tool via `redacting_tool(...)`, then `build_orchestrator_agent` composes the full instructions from four workflow skills + the protocol block + the auto-generated roster.

`tools/data_tools.py` is bound via `init_tools(gateway, catalog, logger)` before the orchestrator runs. The `LocalDataGateway` walks `data_tables/<case_id>/*.csv` and merges `payments_success.csv` + `payments_returns.csv` into a single `payments` table with a synthetic `payment_status` column.

19. The `redacting_tool` Wrapper

Step	What it does	Code
1. Sanitize input	<code>sanitize_message(sub_question)</code> masks PII before the specialist sees it	line 46
2. Lookup history	Read <code>AppContext._specialist_histories[name]</code> ; prepend prior turns for follow-ups	lines 53-83
3. Dedup cache	Key (<code>specialist_name</code> , <code>normalized_subq</code>) against <code>_specialist_call_cache</code> ; return cached result for identical sub-questions	lines 61-78
4. Run inner agent	<code>Runner.run(inner, run_input, context=app_ctx, max_turns=25)</code>	lines 86-89
5. Persist history	<code>histories[name] = result.to_input_list()</code> for next call	lines 110-112
6. Redact output	<code>redact_payload(result.final_output)</code> masks long digit runs on the way back up	line 114
7. MaxTurnsExceeded	If specialist blows 25-turn budget, return structured error string	lines 90-107

20. Inside the `spend_payments` Specialist

The specialist's `instructions` are composed from:

1. `data_query.md` — base playbook: six tools, routing rules, the windowed-answer template, anti-hallucination rules.
2. `spend_payments.md` — domain playbook: four dimensions a pattern answer must cover, exact tool call for each.

Spending-pattern tool call plan

Section	Dimension	Tool call
A.1	Volume per month	<code>summarize_trend('spends', 'Amount', 'Date', period='month', op='sum')</code>
A.2	Txn count per month	<code>summarize_trend('spends', 'Amount', 'Date', period='month', op='count')</code>
B.4	Recurring merchants	<code>summarize_by_group('spends', 'Amount', 'Merchant Name', op='count', top_n=5, sort_by='count')</code>
B.5	High-value merchants	<code>summarize_by_group('spends', 'Amount', 'Merchant Name', op='sum', top_n=5)</code>
B.6	Per-merchant trend	<code>summarize_trend('spends', 'Amount', 'Date', period='month', op='sum', filter_column='Merchant Name', filter_value='<name>') x 3-5</code>
B.7	Industry mix	<code>summarize_by_group('spends', 'Amount', 'Merchant Industry', op='sum', top_n=10)</code>
C.9	Outlier max	<code>aggregate_column('spends', 'Amount', op='max') then query_table(...)</code>
D.11	Spend / successful payments	<code>aggregate_column on spends and payments</code>
D.12	Per-month spend vs. payments	Two <code>summarize_trend</code> calls
D.13	Returned-payment share	<code>aggregate_column('payments', 'Payment Amount', op='sum', filter_column='payment_status', filter_value='return')</code>

Total budget: **11-17 tool calls**, inside the 25-turn cap.

21. Concrete Tool-Call Trace

```

01 list_available_tables()
02 get_table_schema(table="spends")
03 get_table_schema(table="payments")
04 summarize_trend("spends", "Amount", "Date", period="month", op="sum")
05 summarize_trend("spends", "Amount", "Date", period="month", op="count")
06 summarize_by_group("spends", "Amount", "Merchant Name",
                      op="count", top_n=5, sort_by="count")
07 summarize_by_group("spends", "Amount", "Merchant Name", op="sum", top_n=5)
08 summarize_by_group("spends", "Amount", "Merchant Industry", op="sum", top_n=10)
09 summarize_trend("spends", "Amount", "Date", period="month", op="sum",
                  filter_column="Merchant Name", filter_value="S BERTRAM")
10 summarize_trend(..., filter_value="Dependable Plastics")
11 summarize_trend(..., filter_value="AMEXGIFTCARD.COM")
12 aggregate_column("spends", "Amount", op="max")
13 query_table("spends", filter_column="Amount", filter_op="gte",
              filter_value="25000",
              columns="Date,Merchant Name,Merchant Industry,Amount")
14 aggregate_column("spends", "Amount", op="sum")
15 aggregate_column("payments", "Payment Amount", op="sum",
                  filter_column="payment_status", filter_value="success")
16 summarize_trend("payments", "Payment Amount", "payment_date",
                  period="month", op="sum",
                  filter_column="payment_status", filter_value="success")
17 aggregate_column("payments", "Payment Amount", op="sum",
                  filter_column="payment_status", filter_value="return")
--> SpecialistOutput JSON

```

22. Tool-by-Tool Reference

Tool	Location	Spending-pattern role
<code>list_available_tables</code>	<code>data_tools.py:599</code>	Confirms spends , payments , txn_monthly exist for this case
<code>get_table_schema</code>	<code>data_tools.py:714</code>	Gets real column names + canonical/alias map. Must run before filtering on unseen columns
<code>query_table</code>	<code>data_tools.py:918</code>	Row-level fetch with auto-resolved names; returns rows_matching_filter (true count) + sample rows (may be truncated)
<code>aggregate_column</code>	<code>data_tools.py:1151</code>	Returns comma-formatted result that survives <code>\d{6,}</code> redaction. Date-aware fallback for max/min on string-date columns
<code>summarize_trend</code>	<code>data_tools.py:1552</code>	One call = full bucketed series + first/last/peak/trough/total/slope/CV/missing Use for any “shape over time” question
<code>summarize_by_group</code>	<code>data_tools.py:1852</code>	One call = top-N groups + concentration block (HHI, top1/3/5 share). Use for any “shape across category” question

23. Cross-Cutting Concerns

- **Boundary redaction.** Three layers: `sanitize_message` (input to each specialist call), `redact_payload` (output of each specialist call), `_format_aggregate` (thousand separators on all numeric results so values survive the `\d{6,}` mask).
- **Canonical name resolution.** `_resolve_real_table` and `_resolve_real_column` cascade through exact match, catalog aliases, `<name>_data` convention, and normalized fuzzy — so specialists use canonical names from skill `data_hints` while real CSVs may differ.
- **Per-specialist memory.** `AppContext._specialist_histories` lets follow-up questions continue the same specialist thread instead of restarting from cold.
- **Dedup cache.** `AppContext._specialist_call_cache` prevents identical sub-questions from firing twice in the same context.
- **Forced first turn.** `ModelSettings(tool_choice="required")` on both orchestrator and specialists; `reset_tool_choice=True` (SDK default) flips back to "auto" after the first tool call.
- **Pillar cut-off date.** Injected into every specialist prompt; all time-window language is anchored to `cut_off_date`, never today.